

Slowloris Denial-of-Service Attack and Defense: Evaluation Report

Afham Adian

September 19, 2026

1 Introduction

This report documents a controlled experiment in which a web server was subjected to a slow-connection denial-of-service attack, its impact was measured, a defense was applied, and the same attack was repeated to evaluate how effective that defense was.

The experiment involved three roles running at the same time:

- a **legitimate user**, sending normal requests to the server at a steady rate throughout the whole test;
- an **attacker**, generating a large number of malicious slow connections during a fixed attack window;
- a **monitor**, continuously observing the server's health (how many of its connection slots were busy, CPU and memory usage).

Each test ran in three phases: a quiet *baseline* period, an *attack* period, and a *recovery* period after the attack stopped. This was repeated at increasing attack sizes ($N = 20, 50, 100, 200$, where N is the number of connections the attacker attempted to open) so the effect of attack size could be observed directly. The server was configured with a fixed maximum capacity of 30 concurrent connections throughout every test, which is the key number this report's results are measured against.

2 The Attack

2.1 Steps of the Attack

The attack works by exploiting how a web server holds a connection open while it waits for a client to finish sending its request:

1. The attacker opens a large number of connections to the server at once.
2. On each connection, it sends just enough of a request to make it look valid and ongoing, but deliberately never finishes it.
3. Every few seconds, it sends a small, harmless extra fragment on each connection, just enough to convince the server the client is still there and "about to finish," without ever actually completing the request.

4. The server, following normal behaviour, keeps a connection slot reserved for each of these unfinished requests indefinitely, waiting for them to complete.
5. Once enough connections are held open this way, the server has no free slots left for anyone else.

Because each connection is trivial to open and keep alive, this lets a single attacker with modest resources tie up far more server capacity than its own resource usage would suggest.

2.2 Victim-Side Snapshot

To observe the effect directly, the attack was also demonstrated live, following this sequence:

1. Start the server.
2. Open a client terminal.
3. Send a request from the client to the server and confirm a normal response is received.
4. Start the attacker.
5. Send another request from the client terminal: it no longer receives a response.

Figure 1 shows the client terminal's output for the last step: the request hangs until it hits its own timeout, instead of getting a response from the server.

Victim-side terminal

```
afham-adian@afham-adian-Aspire-A715-42G:~/Downloads/Security-Project$ curl -m 10 -i http://127.0.0.1/
curl: (28) Operation timed out after 10002 milliseconds with 0 bytes received
```

Figure 1: **After Attack:** client-side request sent while the attack is running. The request times out with 0 bytes received instead of getting a response.

2.3 Was the Attack Successful?

Yes. Before the attacker was started, the same client request completed normally, as shown in Figure 2; once the attacker was running, the identical request failed as shown earlier in Figure 1.

Victim-side terminal

```
afham-adian@afham-adian-Aspire-A715-42G:~/Downloads/Security-Project$ curl -m 10 -i http://127.0.0.1/
HTTP/1.1 200 OK
Date: Sat, 19 Sep 2026 07:28:10 GMT
Server: Apache/2.4.58 (Ubuntu)
Last-Modified: Fri, 18 Sep 2026 09:36:47 GMT
ETag: "29af-65bba3a142a2"
Accept-Ranges: bytes
Content-Length: 10671
Vary: Accept-Encoding
Content-Type: text/html
```

Figure 2: **Before Attack:** the same client-side request sent before the attacker was started, completing normally with an HTTP 200 OK response.

Table 1 summarizes the legitimate user’s experience during the attack window at each attack size.

Table 1: Legitimate client outcomes during the attack window, undefended server.

Attack size (N)	Client success rate	Avg. latency	Busy slots at peak	Result
20	100%	0.09s	23 / 30	No denial
50	0%	5.01s	30 / 30 (exhausted)	Denial
100	0%	5.01s	30 / 30 (exhausted)	Denial
200	9%	4.55s	30 / 30 (exhausted)	Denial

At $N = 20$, the attack had no visible effect: the server still had free capacity, and the legitimate user’s requests succeeded normally. From $N = 50$ onward, the legitimate user’s success rate collapsed to effectively 0%, and the few requests that did get through took roughly 5 seconds, the point at which the client simply gave up waiting.

2.4 Why the Attack Was Successful

The result lines up exactly with the server’s fixed capacity of 30 concurrent connections. Below that threshold ($N = 20$), the attacker could not occupy enough slots to crowd out the legitimate user. Once the attack size passed that threshold ($N \geq 50$), every available slot was taken up by an unfinished, deliberately-stalled connection, leaving nothing for real traffic. The attack did not need to overwhelm the server computationally: it only needed to exceed the number of connection slots available, which is a comparatively small number and easy to exceed with a modest number of attacking connections.

2.5 Observed Output

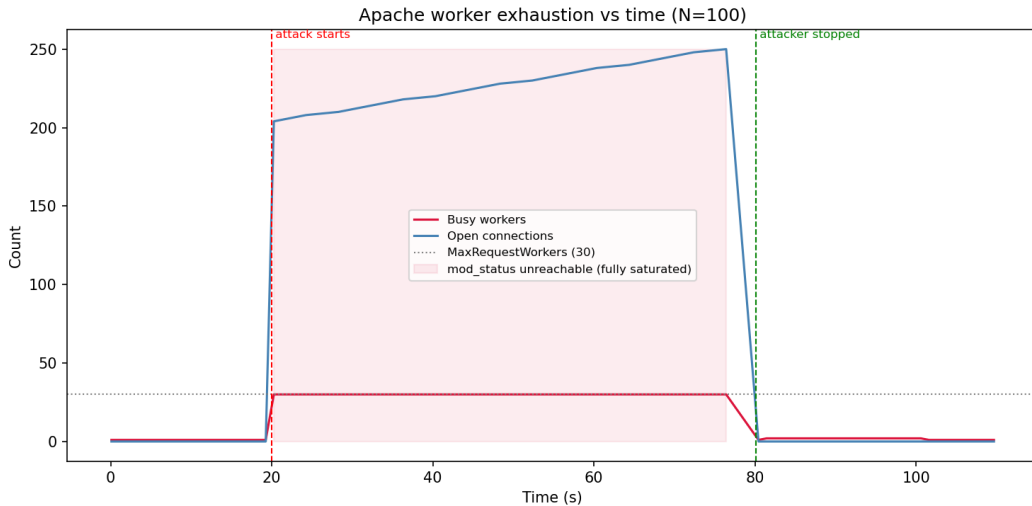


Figure 3: Busy connection slots over time during the $N = 100$ attack. Occupied slots rise to the server’s maximum during the attack window and fall back afterward.

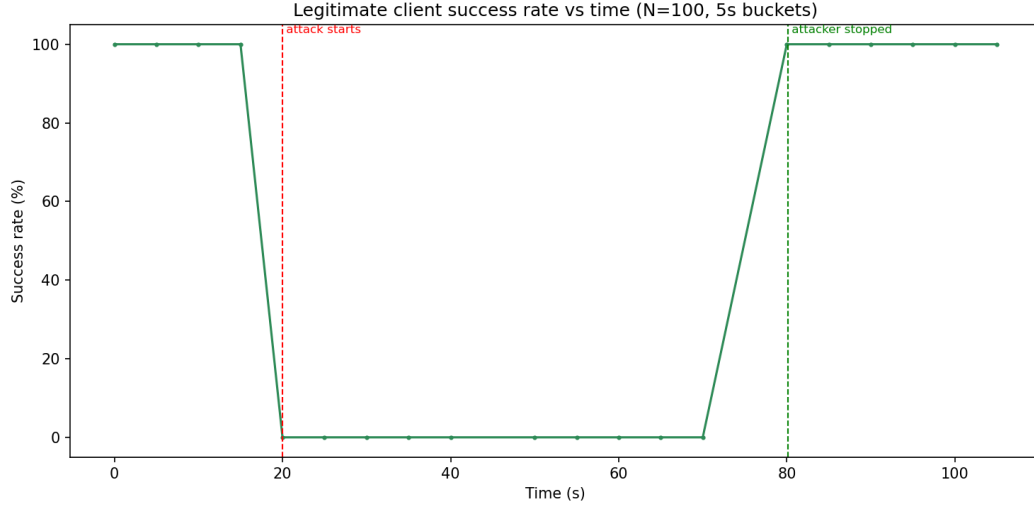


Figure 4: Legitimate client success rate over time during the $N = 100$ attack. Success drops to 0% for the duration of the attack window and recovers immediately once the attack stops.

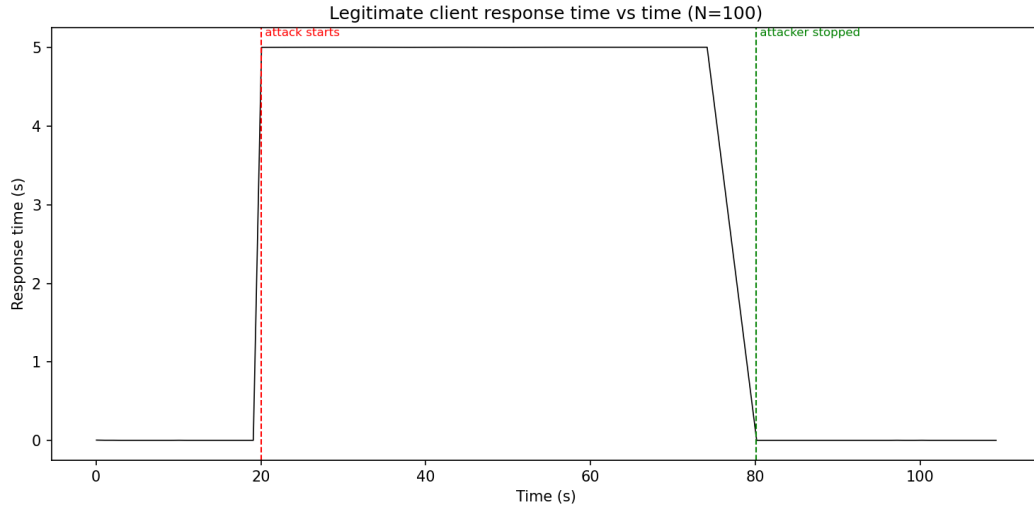


Figure 5: Legitimate client response time over time during the $N = 100$ attack, spiking to the client's own timeout ceiling for the duration of the attack.

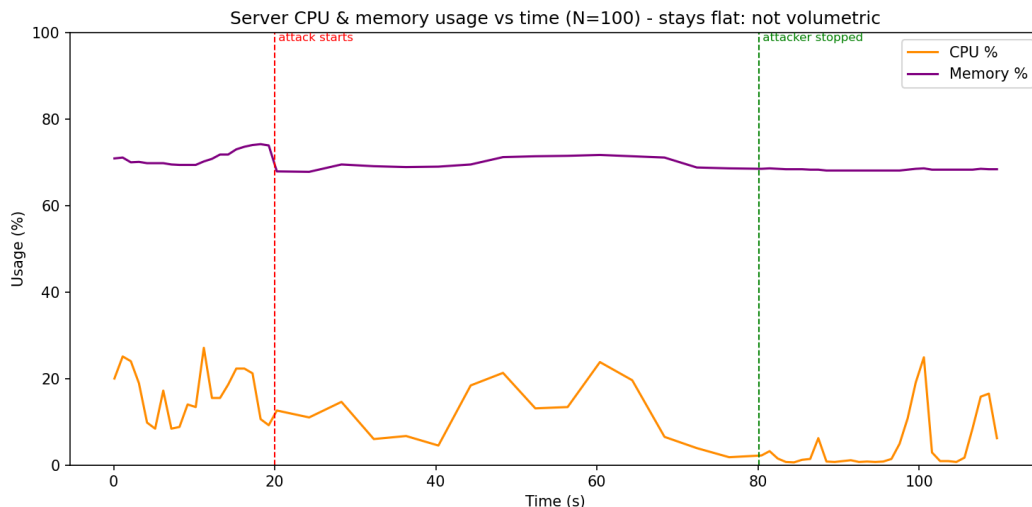


Figure 6: Server CPU and memory usage during the $N = 100$ attack. Usage remains low throughout, showing that the server was not short on computing power, it was short on available connection slots.

3 The Defense

3.1 What Defense Was Applied

Two complementary protective measures were put in place:

1. **A cap on how long any single connection may take to finish sending its request.** If a connection does not complete its request within a bounded time, it is closed and its slot is freed, regardless of what the client does after that.
2. **A limit on how many simultaneous connections a single source may hold open at once.** No matter how many connections one source attempts, only a small number of them are allowed through; the rest are refused outright before they can consume server capacity.

Together, these measures bound how much of the server’s total capacity any one source, legitimate or malicious, can occupy at a time.

In practice, the first measure was implemented by enabling Apache’s request-timeout module with a header read window of 10 to 20 seconds (and a minimum data rate of 500 bytes/second), so a connection sending data too slowly is closed well before it could occupy a slot indefinitely. The second measure was implemented as a firewall rule that permits at most 10 simultaneous connections on port 80 from any single source IP address, rejecting anything beyond that at the network level before it ever reaches Apache.

3.2 Was the Defense Successful?

Yes. Table 2 shows the same measurements as before, this time with the defense active.

Table 2: Legitimate client outcomes during the attack window, defended server.

Attack size (N)	Client success rate	Avg. latency	Busy slots at peak	Result
20	100%	0.05s	12 / 30	No denial
50	100%	0.01s	11 / 30	No denial
100	100%	0.01s	11 / 30	No denial
200	100%	0.00s	10 / 30	No denial

With the defense active, the legitimate user’s success rate stayed at 100% and response times stayed near-instant at every attack size tested, including sizes that had completely denied service before the defense was applied.

3.3 Why the Defense Was Successful

The connection cap meant that, no matter how large the attacker’s intended attack size was, only a small handful of its connections ever actually got through, the rest were rejected immediately. That small number stayed far below the server’s total capacity of 30 slots, so the server always had plenty of room left for legitimate traffic. The request-time cap provided a second layer of protection: even a connection that did get through could only stall for a bounded amount of time before being closed and its slot reclaimed. Because scaling the attack size up further did not increase how many connections actually reached the server, the defended results stay essentially flat across every attack size tested; the defense’s effect does not depend on how big the attack is.

3.4 Observed Output

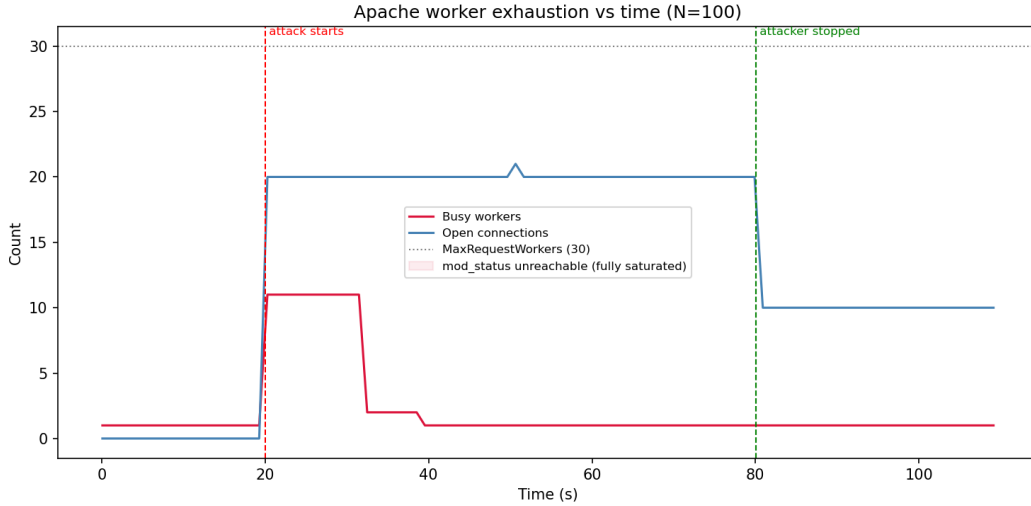


Figure 7: Busy connection slots over time during the $N = 100$ attack, with the defense active. Occupied slots stay low throughout, never approaching the server’s 30-slot limit.

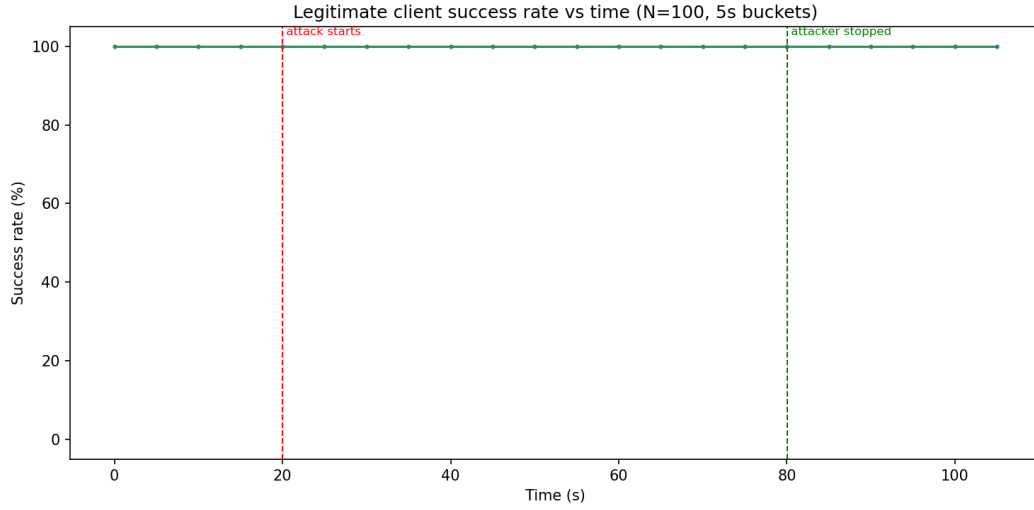


Figure 8: Legitimate client success rate over time during the $N = 100$ attack, with the defense active. Success remains at 100% throughout, including during the attack window.

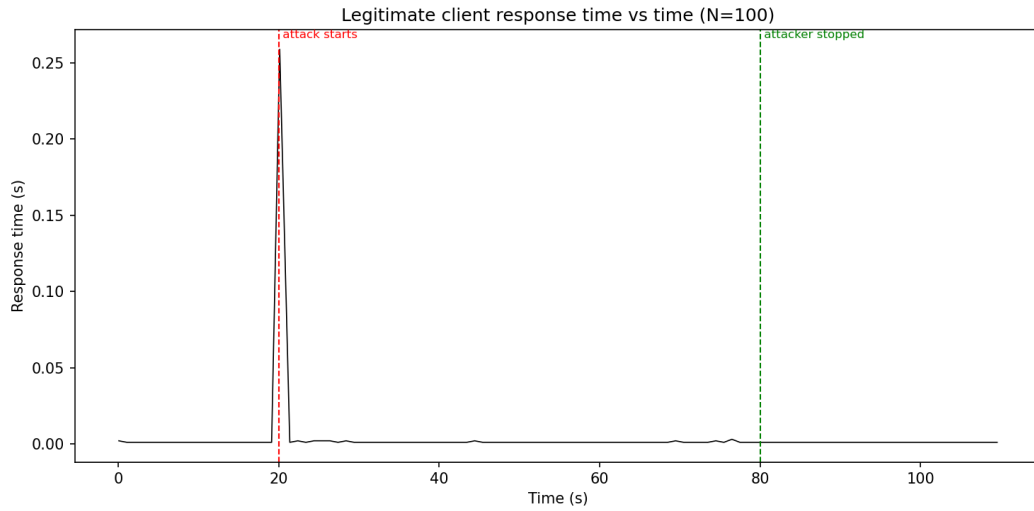


Figure 9: Legitimate client response time over time during the $N = 100$ attack, with the defense active. Response times remain low and stable throughout the attack window.

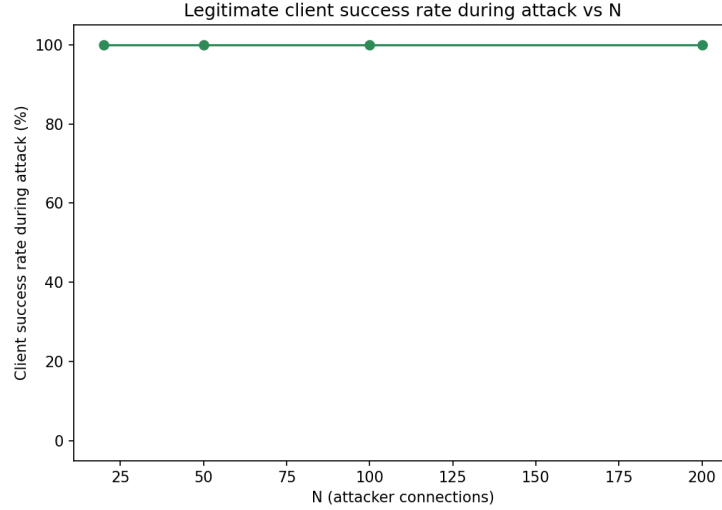


Figure 10: Legitimate client success rate across all tested attack sizes, with the defense active. Success stays at 100% regardless of attack size, in contrast to the undefended results. (The time-to-saturation panel shown for the undefended run is omitted here, since the server never saturated at any tested attack size with the defense active.)

4 Comparison

Figure 11 places the undefended and defended results for the same attack size side by side. Before the defense, exceeding the server’s capacity threshold caused legitimate traffic to fail almost entirely; after the defense, the same attack size had no visible effect on legitimate traffic at all.

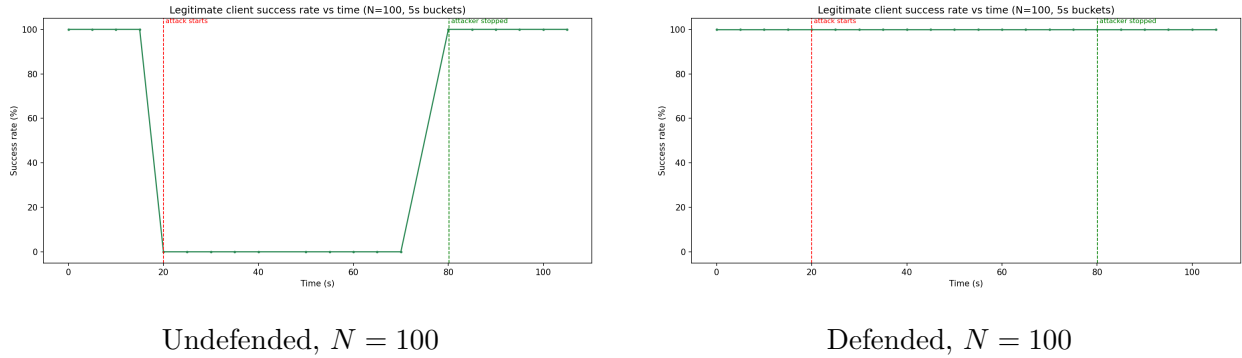


Figure 11: Legitimate client success rate during the same attack size, before and after the defense was applied.

5 Limitations

- The test was run on a single local machine rather than over a real network, so conditions such as multiple attacker locations, real network latency, and other traffic sharing the link were not present.
- The connection-limit defense depends on being able to tell the attacker apart from legitimate

users by their source. Against an attack spread across many different sources at once, this specific measure becomes much harder to apply effectively, though the request-time cap still provides some protection regardless of source.

6 Conclusion

The attack succeeded once its size exceeded the server's spare capacity, causing legitimate users to be denied service almost entirely. Applying a combination of a per-connection time limit and a per-source connection cap prevented any single source from consuming enough of the server's capacity to cause that exhaustion, restoring normal service to legitimate users even under attack sizes that had previously caused a complete denial of service.